One split at a time is often enough: empirical limits of k-sighted decision-tree induction

Mathias Valla[a,*]

[a]*Affiliation to be completed, France*

Abstract

Decision trees are induced by a sequence of local split decisions, although the value of a split depends on the descendants it enables. We study k-sighted induction, a bounded non-greedy compromise between CART-style recursion and globally optimal tree search: candidate axis-aligned splits are scored by the impurity reduction achieved after a depth-k local subtree is optimized. The question is practical rather than existential: does this additional sight materially improve shallow trees and bootstrap forests once fitting time is considered? On 67 PMLB classification datasets, $k = 2$ and $k = 3$ improve mean accuracy over greedy $k = 1$ under this protocol, but by small and uneven margins relative to their cost. For single trees, mean accuracy rises from 0.6931 to 0.7100 and 0.7145, while mean fitting time rises from 0.0076 s to 0.3727 s and 70.90 s. For bootstrap forests, mean accuracy rises from 0.7068 to 0.7125 and 0.7260, while mean fitting time rises from 0.0188 s to 0.6940 s and 96.19 s. A mixed-forest study on 57 datasets shows that a minority of $k = 2$ trees can improve mean accuracy, but the frontier remains a gradual accuracy-time tradeoff. These results support k-sighted induction as a targeted diagnostic or small-data tool, not a default replacement for greedy splitting.

Keywords: Decision trees, Random forests, Greedy algorithms, k-sighted induction, Optimal trees, PMLB

1. Introduction

Most decision-tree learners make one split decision at a time. At each internal node, CART-style induction selects the split that best improves an impurity criterion, then repeats the same operation independently in the child nodes [1, 2]. This greedy rule is fast, stable enough to be useful, and still central to bagging and random forests [3, 4]. It is also myopic: a split is useful because of the subtree it permits, and the locally best split need not be the best first step toward a shallow predictive tree.

This paper asks whether bounded non-greediness is worth its computational price. A k-sighted tree evaluates a candidate split by optimizing a local descendant subtree of depth k, then chooses the root split of the best such local subtree. The case $k = 1$ is ordinary greedy induction. Larger k occupies a middle ground between cheap greedy recursion and exact optimal decision-tree search.

Our question is deliberately narrow and empirical: does multi-step split optimization materially improve shallow decision trees and bootstrap forests when predictive accuracy and fitting time are evaluated jointly? The answer is mixed. Extra sight can improve mean accuracy, and it sometimes improves it substantially on a dataset. Yet the improvement is not consistent, and the fitting time grows by orders of magnitude. The main contribution is a compact benchmark and accuracy-time framing showing why one split at a time remains a strong default.

*Corresponding author.

2. Related work

Greedy top-down tree induction was established by ID3, CART, and related algorithms [2, 1]. Ensembles such as bagging and random forests reduce the variance of greedy trees by averaging many fitted trees [3, 4], but the split decisions inside each tree remain local. This matters for k-sighted forests because additional computation can be spent either inside each split decision or on more trees.

Non-greedy decision-tree induction has a long history. Norton [5] considered downstream tree quality during induction, while Ragavan and Rendell [6] studied descendant-aware feature construction. Murthy and Salzberg [7] warned that even one additional generation of lookahead can be expensive and need not improve accuracy, a theme later analyzed by Elomaa and Malinen [8]. Esmeir and Markovitch [9] cast tree induction as an anytime procedure in which extra computation should be justified by better trees. Norouzi et al. [10] also studied non-greedy tree optimization, emphasizing that non-greediness can enter either split selection or subsequent tree-parameter optimization.

At the other end of the spectrum, optimal-tree methods search directly over tree structures. Hyafil and Rivest [11] showed that optimal binary decision-tree construction is NP-complete. Modern mixed-integer, dynamic-programming, and branch-and-bound methods have made optimal trees practical in selected regimes [12, 13, 14, 15, 16], but scalability remains a central limitation. The k-sighted rule is therefore an intermediate strategy: spend more computation locally, without committing to global tree optimization.

3. k-sighted induction

Let S denote the samples reaching a node, let $I(S)$ be the node impurity, and let $\mathcal{T}_k(s, S)$ be the frontier leaves obtained after applying candidate split s and then optimizing descendant split choices for at most $k-1$ additional generations. A k-sighted split maximizes

$$G_k(s; S) = I(S) - \sum_{L \in \mathcal{T}_k(s,S)} \frac{|L|}{|S|} I(L). \tag{1}$$

For $k = 1$, this is the usual immediate impurity gain. For $k = 2$ or $k = 3$, the current split is chosen according to the best bounded descendant subtree it enables.

This can correct myopic split choices, but it changes the computation sharply. If m split candidates are available at each internal node, a full depth-k binary local subtree has a naive split-configuration count of order m^{2^k-1}. Implementations can cache work and stop early at pure or infeasible nodes, but the qualitative pressure remains: sight depth increases search cost much faster than it increases final tree depth.

To compare accuracy and time, we use a simple utility heuristic. Let $A(k)$ and $T(k)$ be the mean accuracy and mean fitting time at sight depth k. For a user who values accuracy but penalizes multiplicative increases in fitting time, define

$$U_\lambda(k) = A(k) - \lambda \log\{T(k)/T(1)\}, \tag{2}$$

where λ is the accuracy penalty assigned to one natural-log unit of extra fitting time. A larger sight depth beats the greedy baseline under this utility only if

$$A(k) - A(1) > \lambda \log\{T(k)/T(1)\}. \tag{3}$$

The break-even value $\lambda_k^\star = [A(k) - A(1)]/\log\{T(k)/T(1)\}$ is not a full decision model, but it gives a concise accuracy-time scale.

4. Experiments

We evaluated sight depths $k \in \{1, 2, 3\}$ for a decision-tree classifier and a bootstrap forest classifier on 67 PMLB classification datasets [17]. The protocol used one train–test split per dataset: test size 0.25, random state 0, stratification when possible, maximum tree depth 3, all features available at each split, and no explicit cap on split candidates. This is a screening benchmark rather than a full model-selection study: we did not tune hyperparameters or repeat resampling. The main forest benchmark used 100 bootstrap trees with the same k-sighted rule inside every tree; because all features were considered at each split, the comparison isolates sight depth rather than random feature subsampling.

We then studied mixed bootstrap forests on a 57-dataset PMLB sample. Each mixed forest contains proportions p_j of trees with integer sight depths k_j, summarized by an effective sight depth

$$\bar{k} = \sum_j p_j k_j.$$

Table 1: Aggregate performance over 67 PMLB classification datasets.

Estimator	k	Mean acc.	Median acc.	Mean fit (s)
Decision tree	1	0.6931	0.7554	0.0076
Decision tree	2	0.7100	0.7222	0.3727
Decision tree	3	0.7145	0.7333	70.9025
Bootstrap forest	1	0.7068	0.7500	0.0188
Bootstrap forest	2	0.7125	0.7407	0.6940
Bootstrap forest	3	0.7260	0.7612	96.1864

This shorthand is descriptive: for example, a 90% $k = 1$ and 10% $k = 2$ forest and a 95% $k = 1$ and 5% $k = 3$ forest both have $\bar{k} = 1.10$, but very different costs. We evaluated all curves at 20, 40, 60, 100, and 200 trees.

5. Results

Table 1 gives the aggregate benchmark. Extra sight improves mean accuracy, but the improvement is thin relative to the time increase. For single trees, $k = 2$ adds 1.69 percentage points over $k = 1$ while fitting time is about 49 times larger; $k = 3$ adds 2.15 points while fitting time is about 9.4×10^3 times larger. For bootstrap forests, $k = 2$ adds 0.57 points at about 37 times the fitting time, and $k = 3$ adds 1.92 points at about 5.1×10^3 times the fitting time.

Figure 1 shows the same pattern visually. Accuracy grows mildly as k increases, while fitting time grows on a logarithmic scale. The break-even $\lambda_k^\star$ values are 0.00435 for tree $k = 2$ versus $k = 1$, 0.00235 for tree $k = 3$ versus $k = 1$, 0.00158 for forest $k = 2$ versus $k = 1$, and 0.00225 for forest $k = 3$ versus $k = 1$. Thus any time penalty larger than roughly 0.16–0.44 accuracy percentage points per natural-log unit of fitting time would favor greedy induction in these aggregate comparisons.

The gains are not uniform across datasets. Counting ties as wins, $k = 1$ was tied for the best single-tree accuracy on 30 datasets, $k = 2$ on 31 datasets, and $k = 3$ on 37 datasets. For forests, the corresponding counts were 36, 29, and 29. These descriptive counts are not significance tests; they reinforce the central point that larger k can alter the average, but it does not dominate greedy splitting.

5.1. Mixed forests

Figure 2 shows the full forest-size grid: the left panel plots mean accuracy against the number of trees, and the right panel plots the same grid against mean fitting time on a log scale. The grid asks whether a mostly greedy forest benefits from a few farther-sighted trees. It shows a limited benefit rather than a shortcut: $k = 2$ minorities often improve the mean, but the improvement follows the same accuracy-time frontier. At 100 trees, the greedy forest has mean accuracy 0.7241 and mean fitting time 0.453 s. Replacing 5%, 10%, 25%, and 50% of trees by $k = 2$ trees raises mean accuracy to 0.7346, 0.7363, 0.7458, and 0.7504, while increasing mean fitting time by factors of 2.3, 3.5, 7.3, and 13.6. A pure $k = 2$ 100-tree forest reaches 0.7536 accuracy at 26.2 times the greedy fitting time.

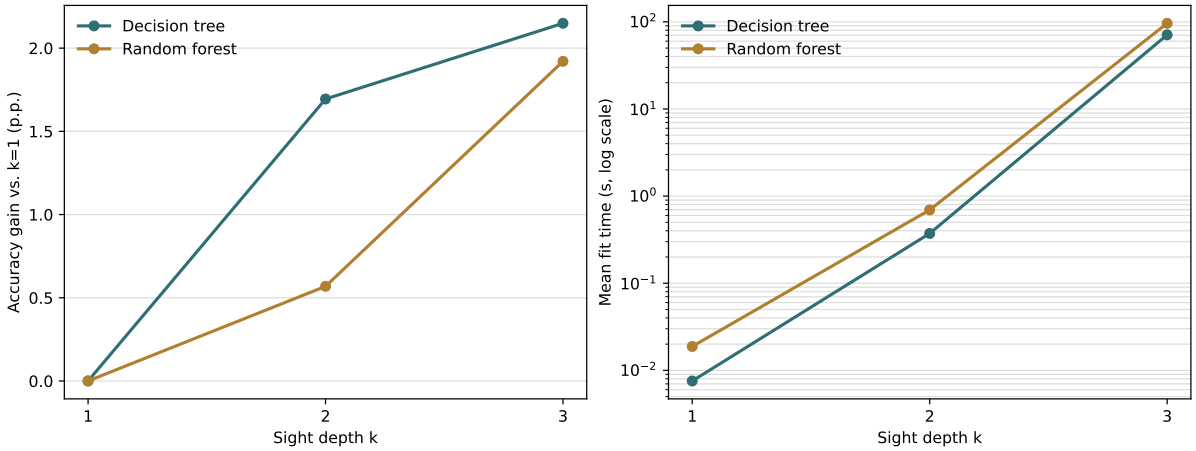

Figure 1: Mean accuracy gain and mean fitting time as sight depth increases from $k = 1$ to $k = 3$. Accuracy gains are measured relative to the greedy $k = 1$ baseline.

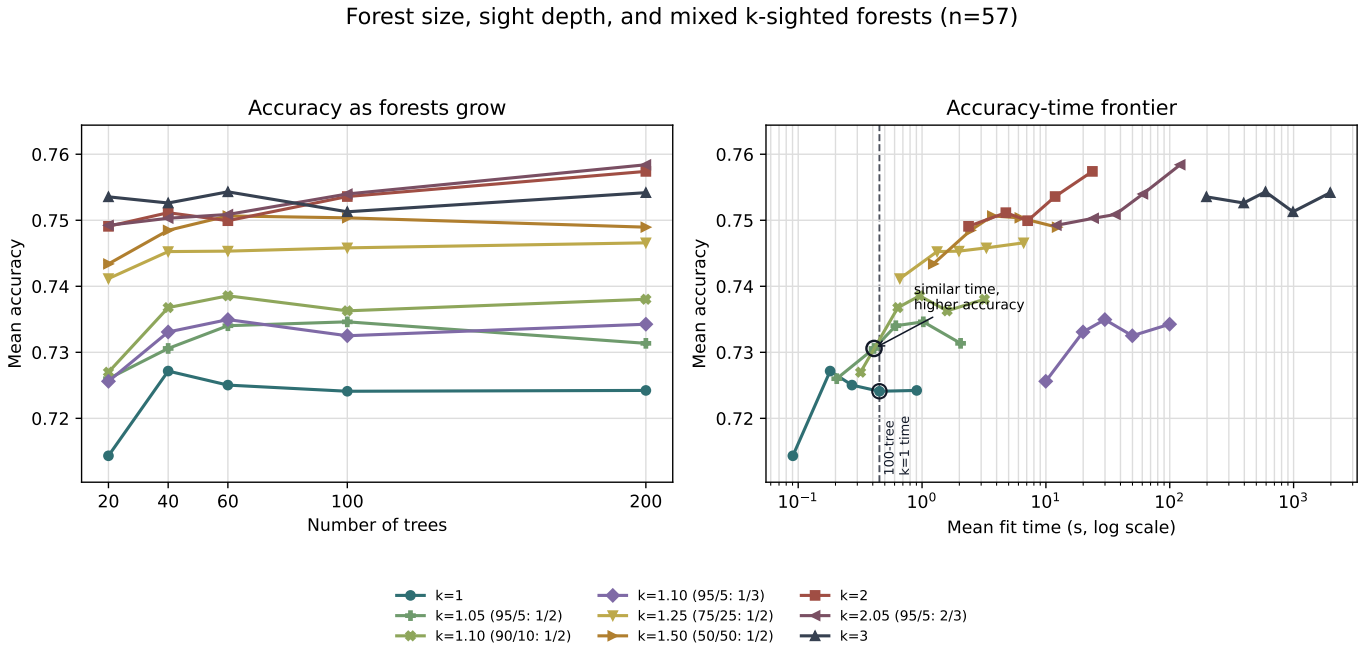

Figure 2: Forest-size grid for pure and mixed k-sighted bootstrap forests on the 57-dataset mixed-forest benchmark. Left: mean accuracy versus number of trees. Right: the same curve points versus mean fitting time on a logarithmic scale; the dashed vertical line marks the mean fitting time of the 100-tree greedy forest. Each curve is evaluated at 20, 40, 60, 100, and 200 trees. Legend labels use effective sight $\bar{k}$, the average tree sight depth in the ensemble; the two $\bar{k} = 1.10$ curves are distinct mixtures, one with 90% $k = 1$ and 10% $k = 2$ trees and one with 95% $k = 1$ and 5% $k = 3$ trees.

The right panel also shows why mixed forests can be beneficial. The dashed vertical line marks the fitting-time budget of the 100-tree greedy forest. Near that same budget, a 40-tree forest with 95% $k = 1$ trees and 5% $k = 2$ trees has higher mean accuracy (0.7306 versus 0.7241) while taking slightly less mean fitting time (0.409 s versus 0.453 s). Thus the practical question is not only whether every tree should be farther-sighted. A small number of $k = 2$ trees can sometimes move an otherwise greedy ensemble to a better accuracy-time point.

Adding $k = 3$ trees is much more expensive. At 100 trees, a 95% $k = 1$ and 5% $k = 3$ forest reaches 0.7325 accuracy but costs 110 times the 100-tree greedy forest. A 95% $k = 2$ and 5% $k = 3$ forest reaches 0.7540 accuracy but costs 134 times the greedy forest. Within the evaluated grid, increasing the number of greedy trees alone does not close the gap on this sample: a 200-tree greedy forest has mean accuracy 0.7242, essentially unchanged from the 100-tree greedy forest, whereas a 20-tree $k = 2$ forest reaches 0.7491 at 5.2 times the 100-tree greedy fitting time. The pure $k = 3$ curve is still farther to the right and does not dominate the $k = 2$ frontier. The lesson is not that greedy trees always suffice. It is that improvements from extra sight arrive through a visible and expensive accuracy-time frontier, not through a nearly free ensemble effect.

6. Discussion

The empirical message is conservative. k-sighted induction is a plausible correction to greedy myopia, and the mixed-forest results show that a small number of $k = 2$ trees can improve a shallow forest. But the gains are uneven, and the computation grows much faster than accuracy. These comparisons should be read as controlled benchmarks, not as a complete ranking of ensemble design choices: they use one train–test split per dataset, shallow trees, all features at each split, and wall-clock fitting times from this implementation. Bagging and random forests already provide cheap ways to reduce instability by averaging many greedy trees; spending computation inside every split decision must clear a high bar.

These results do not rule out k-sighted trees. They identify where the method is most plausible: small datasets, shallow interpretable trees, specialized interaction structures, or settings where fitting time is cheap relative to errors. They also suggest that $k = 2$ is the natural first target for any practical non-greedy extension. In contrast, $k = 3$ looks difficult to justify as a routine default under this protocol.

7. Conclusion

Multi-step split optimization can improve decision trees and forests, but in this benchmark it does so by a narrow and inconsistent margin relative to its computational cost. Greedy CART-style induction is not globally optimal; nevertheless, it remains a remarkably competitive accuracy-time compromise. For routine shallow tabular trees and forests, one split at a time is often enough.

CRediT authorship contribution statement

Mathias Valla: Conceptualization, Methodology, Software, Investigation, Writing–original draft.

Declaration of competing interest

The author declares no competing interests.

Data and code availability

The benchmark datasets are available from PMLB [17]. The open-source implementation, scripts, tables, and an experimental package for spherical tree and k-sighted tree experiments will be available in the project GitHub repository.

Funding

Funding information is to be completed before submission.

References

[1] L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone, Classification and Regression Trees, Wadsworth, Belmont, CA, 1984.

[2] J. R. Quinlan, Induction of decision trees, Machine Learning 1 (1) (1986) 81–106. doi:10.1007/BF00116251.

[3] L. Breiman, Bagging predictors, Machine Learning 24 (2) (1996) 123–140. doi:10.1007/BF00058655.

[4] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[5] S. W. Norton, Generating better decision trees, in: Proceedings of the Eleventh International Joint Conference on Artificial Intelligence, 1989, pp. 800–805.
URL https://www.ijcai.org/Proceedings/89-1/Papers/128.pdf

[6] H. Ragavan, L. A. Rendell, Lookahead feature construction for learning hard concepts, in: Machine Learning Proceedings 1993, Morgan Kaufmann, 1993, pp. 252–259. doi:10.1016/B978-1-55860-307-3.50039-3.

[7] S. K. Murthy, S. L. Salzberg, Lookahead and pathology in decision tree induction, in: Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence, 1995, pp. 1025–1031.
URL https://www.ijcai.org/Proceedings/95-2/Papers/002.pdf

[8] T. Elomaa, T. Malinen, On look-ahead and pathology in decision tree learning, Journal of Experimental & Theoretical Artificial Intelligence 17 (1–2) (2005) 19–33. doi:10.1080/09528130512331315936.

[9] S. Esmeir, S. Markovitch, Anytime learning of decision trees, Journal of Machine Learning Research 8 (2007) 891–933.
URL https://www.jmlr.org/papers/v8/esmeir07a.html

[10] M. Norouzi, M. D. Collins, M. A. Johnson, D. J. Fleet, P. Kohli, Efficient non-greedy optimization of decision trees, in: Advances in Neural Information Processing Systems, Vol. 28, 2015, pp. 1729–1737.

[11] L. Hyafil, R. L. Rivest, Constructing optimal binary decision trees is NP-complete, Information Processing Letters 5 (1) (1976) 15–17. doi:10.1016/0020-0190(76)90095-8.

[12] D. Bertsimas, J. Dunn, Optimal classification trees, Machine Learning 106 (7) (2017) 1039–1082. doi:10.1007/s10994-017-5633-9.

[13] S. Verwer, Y. Zhang, Learning optimal classification trees using a binary linear program formulation, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 1625–1632. doi:10.1609/aaai.v33i01.33011624.

[14] X. Hu, C. Rudin, M. Seltzer, Optimal sparse decision trees, in: Advances in Neural Information Processing Systems, Vol. 32, 2019, pp. 7265–7273.

[15] J. Lin, C. Zhong, D. Hu, C. Rudin, M. Seltzer, Generalized and scalable optimal sparse decision trees, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 6150–6160.
URL https://proceedings.mlr.press/v119/lin20g.html

[16] S. Aghaei, A. Gómez, P. Vayanos, Strong optimal classification trees, Operations Research 73 (4) (2025) 2223–2241. doi:10.1287/opre.2021.0034.

[17] R. S. Olson, W. La Cava, P. Orzechowski, R. J. Urbanowicz, J. H. Moore, PMLB: a large benchmark suite for machine learning evaluation and comparison, BioData Mining 10 (2017) 36. doi:10.1186/s13040-017-0154-4.